Status	Finished
Started	Tuesday, 15 September 2026, 3:47 PM
Completed	Tuesday, 22 September 2026, 4:05 PM
Duration	7 days
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

The program must accept **N** series of keystrokes as string values as the input. The character **^** represents undo action to clear the last entered keystroke. The program must print the string typed after applying the undo operations as the output. If there are no characters in the string then print **-1** as the output.

Boundary Condition(s):

1 <= N <= 100
1 <= Length of each string <= 100

Input Format:

The first line contains the integer N.
The next N lines contain a string on each line.

Output Format:

The first N lines contain the string after applying the undo operations.

Example Input/Output 1:

Input:

3
Hey ^ goooo^^glee^
lucke^y ^charr^ms
ora^^nge^^^^

Output:

Hey google
luckycharms
-1

Answer: (penalty regime: 0 %)

```
n=int(input())

for _ in range(n):
    s=input()
    stack=[]

    for ch in s:
        if ch == '^':
            if stack:
                stack.pop()
        else:
            stack.append(ch)

    if stack:
        print(''.join(stack))
```

```
    else:
        print(-1)

```

	Input	Expected	Got	
✓	3 Hey ^ goooo^^glee^ lucke^y ^charr^ms ora^^nge^^^^	Hey google luckycharms -1	Hey google luckycharms -1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

Open brackets must be closed by the same type of brackets.

Open brackets must be closed in the correct order.

Constraints:

1 <= s.length <= 10^4

s consists of parentheses only '()[]{}'.

For example:

Test	Result
`print(ValidParenthesis("()"))`	`true`
`print(ValidParenthesis("()[]{}"))`	`true`
`print(ValidParenthesis("(]"))`	`false`

Answer: (penalty regime: 0 %)

Reset answer

```
def ValidParenthesis(s):
    # Map closing brackets to their matching opening brackets
    mapping = {")": "(", "}": "{", "]": "["}
    stack = []

    for char in s:
        if char in mapping:
            # Pop the top element if stack is not empty, otherwise use a dummy character
            top = stack.pop() if stack else '#'
            if mapping[char] != top:
                return "false"
        else:
            # It's an opening bracket, push it onto the stack
            stack.append(char)

    # If the stack is empty, all brackets were correctly closed
    return "true" if len(stack) == 0 else "false"


```

	Test	Expected	Got	
✓	`print(ValidParenthesis("()"))`	`true`	`true`	✓
✓	`print(ValidParenthesis("()[]{}"))`	`true`	`true`	✓
✓	`print(ValidParenthesis("(]"))`	`false`	`false`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Given a **non-empty** string s and an abbreviation abbr, return whether the string matches with the given abbreviation.

A string such as "word" contains only the following valid abbreviations:

["word", "1ord", "w1rd", "wo1d", "wor1", "2rd", "w2d", "wo2", "1o1d", "1or1", "w1r1", "1o2", "2r1", "3d", "w3", "4"]

Notice that only the above abbreviations are valid abbreviations of the string "word". Any other string is not a valid abbreviation of "word".

Note:

Assume s contains only lowercase letters and abbr contains only lowercase letters and digits.

Example 1:

Input

internationalization
i12iz4n

Output

true

Explanation

Given **s** = "internationalization", **abbr** = "i12iz4n":

Return true.

Example 2:

Input

apple
a2e

Output

false

Explanation

Given **s** = "apple", **abbr** = "a2e":

Return false.

Answer: (penalty regime: 0 %)

```python
import sys

def check(s: str, abbr: str) -> str:
    i, j = 0, 0
    m, n = len(s), len(abbr)

    while i < m and j < n:
        if abbr[j].isdigit():
            if abbr[j] == '0':
                return "false"
            num = 0
            while j < n and abbr[j].isdigit():
                num = num * 10 + int(abbr[j])
                j += 1
            i += num
        else:
            if s[i] != abbr[j]:
                return "false"
            i += 1
            j += 1

    return "true" if i == m and j == n else "false"

def main():
    # Read all input lines and strip surrounding whitespace/newlines
    lines = [ln.strip() for ln in sys.stdin if ln.strip()]

    # Process inputs in pairs (String, Abbreviation)
    for idx in range(0, len(lines), 2):
        if idx + 1 < len(lines):
            print(check(lines[idx], lines[idx+1]))

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	internationalization i12iz4n	true	true	✓
✓	apple a2e	false	false	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Assume that the given string has enough memory.

Don't use any extra space(IN-PLACE)

Sample Input 1

a2b4c6

Sample Output 1

aabbbbccccccc

Answer: (penalty regime: 0 %)

```
import sys
import re

def expand_string(s: str) -> str:
    # Use regular expressions to find all pairs of (letters)(digits)
    # This safely handles single digits like '2' and multi-digits like '12'
    pairs = re.findall(r'([a-zA-Z])(\d+)', s)

    # Reconstruct the string without creating tracking structures
    result = [char * int(count) for char, count in pairs]
    return "".join(result)

def main():
    # Read all lines from standard input to handle any test cases
    for line in sys.stdin:
        clean_line = line.strip()
        if clean_line:
            print(expand_string(clean_line))

if __name__ == "__main__":
    main()


```

	Input	Expected	Got	
✓	a2b4c6	aabbbbccccccc	aabbbbccccccc	✓
✓	a12b3d4	aaaaaaaaaaaabbbdddd	aaaaaaaaaaaabbbdddd	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

A pangram is a sentence where every letter of the English alphabet appears at least once.

Given a string sentence containing only lowercase English letters, return true if sentence is a pangram, or false otherwise.

Example 1:

Input:

thequickbrownfoxjumpsoverthelazydog

Output:

true

Explanation: sentence contains at least one of every letter of the English alphabet.

Example 2:

Input:

arvijayakumar

Output: false

Constraints:

1 <= sentence.length <= 1000

sentence consists of lowercase English letters.

For example:

Test	Result
`print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))`	`true`
`print(checkPangram('arvijayakumar'))`	`false`

Answer: (penalty regime: 0 %)

Reset answer

```
def checkPangram(s):
    # Use a set to collect all unique characters from the string
    unique_letters = set(s)

    # Check if the number of unique letters equals the 26 letters of the alphabet
    if len(unique_letters) == 26:
        return "true"
    else:
        return "false"
```

	Test	Expected	Got	
✓	`print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))`	`true`	`true`	✓
✓	`print(checkPangram('arvijayakumar'))`	`false`	`false`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Consider the below words as key words and check the given input is key word or not.

keywords: {break, case, continue, default, defer, else, for, func, goto, if, map, range, return, struct, type, var}

Input format:

Take string as an input from stdin.

Output format:

Print the word is key word or not.

Example Input:

break

Output:

break is a keyword

Example Input:

IF

Output:

IF is not a keyword

For example:

Input	Result
break	break is a keyword
IF	IF is not a keyword

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Define the set of valid keywords
    keywords = {
        "break", "case", "continue", "default", "defer", "else",
        "for", "func", "goto", "if", "map", "range",
        "return", "struct", "type", "var"
    }

    # Read all lines from standard input
    for line in sys.stdin:
        # Strip trailing newlines and spaces
        word = line.strip()
        if not word:
            continue

        # Check if the word exists in the keyword set (case-sensitive)
        if word in keywords:
            print(f"{word} is a keyword")
        else:
            print(f"{word} is not a keyword")

if __name__ == "__main__":
    main()

```

	Input	Expected	Got	
✓	break	break is a keyword	break is a keyword	✓
✓	IF	IF is not a keyword	IF is not a keyword	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Find if a String2 is substring of String1. If it is, return the index of the first occurrence. else return -1.

Sample Input 1

thistest123string

123

Sample Output 1

8

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read all lines from standard input and filter empty values
    lines = [ln.strip() for ln in sys.stdin if ln.strip()]

    # Process inputs in pairs (String1, String2)
    for idx in range(0, len(lines), 2):
        if idx + 1 < len(lines):
            string1 = lines[idx]
            string2 = lines[idx + 1]

            # .find() returns the index of the first occurrence, or -1 if not found
            print(string1.find(string2))

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	thistest123string 123	8	8	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Given a string, determine if it is a palindrome, considering only alphanumeric characters and ignoring cases.

Note: For the purpose of this problem, we define empty string as valid palindrome.

Example 1:

```
Input:
A man, a plan, a canal: Panama

Output:
1
```

Example 2:

```
Input:
race a car

Output:
0
```

Constraints:

- `s` consists only of printable ASCII characters.

Answer: (penalty regime: 0 %)

```
import sys

def is_palindrome(s: str) -> int:
    # Filter out non-alphanumeric characters and lowercase everything
    filtered_chars = [char.lower() for char in s if char.isalnum()]

    # An empty string or a string matching its reverse is a valid palindrome
    if filtered_chars == filtered_chars[::-1]:
        return 1
    return 0

def main():
    # Read the full input block from standard input
    # splitlines() ensures we handle cases if the platform provides input per line
    lines = sys.stdin.read().splitlines()
    for line in lines:
        if line.strip() or line == "":  # Handle empty string validation rule
            print(is_palindrome(line))

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	A man, a plan, a canal: Panama	1	1	✓
✓	race a car	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Write a Python program to get one string and reverses a string. The input string is given as an array of characters `char[]`.

You may assume all the characters consist of printable ascii characters.

Example 1:

```
Input:
hello
Output:
olleh
```

Example 2:

```
Input:
Hannah
Output:
hannaH
```

Answer: (penalty regime: 0 %)

```
import sys

def reverse_string(char_array: list[str]) -> None:
    left = 0
    right = len(char_array) - 1

    while left < right:
        # Swap the characters in-place
        char_array[left], char_array[right] = char_array[right], char_array[left]
        left += 1
        right -= 1

def main():
    # Read lines directly from standard input
    for line in sys.stdin:
        # Strip trailing newlines/spaces
        clean_line = line.rstrip('\r\n')
        if not clean_line:
            continue

        # Convert the string to a character array and reverse it
        char_list = list(clean_line)
        reverse_string(char_list)

        # Print ONLY the final reversed string
        print(''.join(char_list))

if __name__ == "__main__":
    main()

```

	Input	Expected	Got	
✓	hello	olleh	olleh	✓
✓	Hannah	hannaH	hannaH	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Given a string S which is of the format USERNAME@DOMAIN.EXTENSION, the program must print the EXTENSION, DOMAIN, USERNAME in the reverse order.

Input Format:

The first line contains S.

Output Format:

The first line contains EXTENSION.
The second line contains DOMAIN.
The third line contains USERNAME.

Boundary Condition:

1 <= Length of S <= 100

Example Input/Output 1:

Input:

abcd@gmail.com

Output:

com
gmail
abcd

For example:

Input	Result
arvijayakumar@rajalakshmi.edu.in	edu.in rajalakshmi arvijayakumar

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read the full input string from standard input
    s = sys.stdin.read().strip()
    if not s:
        return

    # 1. Split into username and the rest at the '@' symbol
    username, rest = s.split('@', 1)

    # 2. Split the rest at the very first dot to isolate the domain from the extension
    domain, extension = rest.split('.', 1)

    # 3. Print in the requested reverse order
    print(extension)
    print(domain)
    print(username)

if __name__ == "__main__":
    main()

```

	Input	Expected	Got	
✓	abcd@gmail.com	com gmail abcd	com gmail abcd	✓
✓	arvijayakumar@rajalakshmi.edu.in	edu.in rajalakshmi arvijayakumar	edu.in rajalakshmi arvijayakumar	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.